

CSCE 608 DATABASE SYSTEMS · MODULE 06 · LECTURE NOTES

Sorting and Joins

The two operations that dominate query cost.

MODULE OUTCOMES

- Implement external merge sort and compute its I/O cost.
- Implement nested loop, sort-merge, and hash joins.
- Analyse each join's I/O cost and say when each wins.
- Explain why hash joins usually win and when they do not.
- Explain how joins degrade when memory is insufficient.

1 External sorting

Sorting underlies ORDER BY, GROUP BY, DISTINCT, sort-merge join, and index construction. Doing it on data larger than memory is a foundational problem.

TWO PHASES, BOTH SEQUENTIAL

Phase 1 — run generation. Read B pages into the buffer pool, sort them in memory with any in-memory algorithm, and write the sorted result back out as a *run*. Repeat until the input is exhausted. This produces $\lceil N/B \rceil$ sorted runs.

Phase 2 — merging. Merge $B-1$ runs simultaneously, using one buffer page as the input window for each run and one for output. When a run's buffer empties, read its next page. Repeat until one run remains.

Every I/O in both phases is sequential. Runs are written sequentially and read sequentially; output is written sequentially. **This is the entire point** — Module 02 established that sequential access is 10 to 100 times faster, and this algorithm is constructed to use nothing else.

With a realistic buffer pool, one merge pass almost always suffices, so the practical cost is 'read everything twice, write everything twice'.

$$\text{passes} = 1 + \lceil \log_{B-1}(N/B) \rceil$$

$$\text{total I/O} = 2N \times \text{passes}$$

N (pages)	B (buffer pages)	Runs	Passes	Total I/O
1,000	100	10	2	4,000
10^6	1,000	1,000	2	4×10^6
10^9	1,000	10^6	3	6×10^9

N (pages)	B (buffer pages)	Runs	Passes	Total I/O
10^9	10,000	10^5	2	4×10^9

The base of the logarithm is the buffer size, which is why memory helps so disproportionately here. At $B = 1000$ the logarithm has base 999, so sorting a terabyte takes two or three passes, not forty. Note the last two rows: ten times the memory removes an entire pass over a billion pages.

- **Replacement selection** generates runs averaging $2B$ pages rather than B , by keeping a heap in memory and emitting the smallest record still greater than the last one written. Fewer, longer runs mean a lower merge fanout requirement.
- **Double buffering** overlaps I/O with computation: while one buffer page is being merged, the next is being read. Without it the CPU idles through every read.
- **Skip the sort entirely** when a B+tree already provides the required order. Scanning an index is usually cheaper than sorting — and the optimiser must know this, which is Module 08's 'interesting orders'.
- **Top-K for LIMIT**. `ORDER BY x LIMIT 10` does not need a sort; a bounded heap of ten elements over a single scan suffices, and the difference on a large table is enormous.
- **Normalised keys**. Encode the sort key as a byte string such that byte-wise comparison gives the correct order, then compare with `memcmp`. Once I/O is sequential, comparison cost dominates, and this is a large constant-factor win that the asymptotic analysis does not show.

2 Joins

Nested loop variant	Cost	Assessment
Naive (tuple-at-a-time)	$M + (m \times N)$ — scan the inner relation once per outer row.	Pathological. Never correct; present only to motivate the others.
Block nested loop	$M + (M \times N)$ — scan the inner once per outer <i>page</i> , or better, per block of buffer.	Acceptable when the outer relation is small, and the only option for a non-equi join such as an inequality or a range overlap.
Index nested loop	$M + (m \times \text{cost of one index lookup})$.	Excellent when the outer relation is small and the inner is indexed on the join key. This is the right plan far more often than its reputation suggests — it is the standard plan for OLTP queries that join a handful of rows to a large indexed table.

SORT-MERGE JOIN

Sort both inputs on the join key (§1), then advance through them together in a single coordinated pass, emitting matches.

Cost is the two sorts plus $M + N$ for the merge. The sorts dominate.

Which makes it free when an input is already sorted — because it arrived from a clustered index scan, or because an earlier sort-merge join in the same plan produced it in that order. The optimiser tracks this as an **interesting order** (Module 08), and it is why a plan may choose sort-merge over hash despite a higher local cost.

And the output is sorted, which may satisfy a subsequent ORDER BY, GROUP BY, or merge join for nothing. Accounting for that benefit is one of the genuinely subtle parts of optimisation.

```

PHASE 1 -- PARTITION both relations by hash(join_key) % P
R -> R0 R1 ... RP      S -> S0 S1 ... SP
Matching rows now share a partition index.
Cost: read and write both relations once.

PHASE 2 -- JOIN partition by partition
for i in 0..P:
    build an in-memory hash table on Ri (the smaller side)
    probe it with every row of Si
Cost: read both relations once.

TOTAL: 3(M + N)          EQUI-JOINS ONLY

```

Partitioning is what makes the second phase a single pass: two rows can only join if their keys are equal, and equal keys hash to the same partition, so a row in R_i can only match rows in S_i . Each partition is sized to fit in memory, so each is joined with one in-memory hash table.

The restriction to equi-joins is fundamental. A hash value tells you nothing about ordering, so hashing cannot help with $<$, $>$, or range overlap predicates. This is why sort-merge and block nested loop remain necessary.

Algorithm	I/O cost	Requires	Best when
Index nested loop	$M + m \cdot \text{lookup}$	An index on the inner join key.	Small outer, indexed inner. The OLTP default.
Block nested loop	$M + M \cdot N$	Nothing.	A very small outer relation, or a non-equi join where nothing else applies.
Sort-merge	sorts + $M + N$	An orderable join key.	An input is already sorted , or the sorted output is wanted downstream.
Hash	$3(M + N)$	An equi-join.	Large unsorted inputs. The default for analytical work.

3 When memory runs out

SPILLING IS A CLIFF, NOT A SLOPE

A hash join sizes its build-side hash table according to the optimiser's estimate of how many rows the build side will produce.

If that estimate is wrong — and Module 08 explains why it frequently is — the hash table does not fit in the memory allocated. The join must then spill partitions to disk and process them in multiple batches.

The cost does not degrade smoothly. It jumps, often by an order of magnitude, as soon as the threshold is crossed.

This is the usual explanation for a query that was fast last week and is slow today. Nothing changed in the query; the table grew past the point where the estimate and the memory limit were still compatible, and the plan fell off the cliff.

Recursive partitioning handles the general case — partition an oversized partition again — **but data skew can defeat it entirely.** If a single key value accounts for millions of rows, every one of them hashes to the same partition, and no amount of repartitioning separates them.

- **EXPLAIN ANALYZE shows estimated against actual row counts at every node.** A large discrepancy is the single most informative diagnostic available, and it is where every investigation should start.
- **Look for spills.** PostgreSQL reports Batches: 4 on a hash join and disk usage on a sort. **More than one batch means the estimate was wrong** and the join is paying the cliff.
- **Look for a nested loop over a large outer relation.** This almost always indicates a cardinality underestimate — the optimiser chose nested loop because it expected a handful of rows.
- **Check for skew.** Query the distribution of the join key. One dominant value defeats hash partitioning and makes the estimated cost meaningless.
- **Then fix the estimate, not the plan.** Update statistics, add an extended statistics object for correlated columns, or rewrite the predicate so it can be estimated. **Hints are a last resort,** because they freeze a decision that was correct for today's data and will be wrong later — and unlike a bad estimate, a hint never corrects itself.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
CMU 15-445 — Lectures 11–12, Sorting and Joins	External sort, the three join families, and the cost analysis of §2.

Resource	What it covers for this module
Graefe — Query Evaluation Techniques for Large Databases (free)	The comprehensive survey of sorting, hashing, and join processing. Dense, and still the reference.
Shapiro — Join Processing in Database Systems with Large Main Memories (free)	The Grace and hybrid hash join analysis of §2.
PostgreSQL — Using EXPLAIN	The diagnostic material in §3, with worked examples. Read this before tuning anything.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Implement external merge sort. Verify the I/O count against the formula for several values of N and B .
- 2 Plot passes against buffer size for a fixed N and confirm the step structure predicted by the logarithm.
- 3 Implement replacement selection and measure the average run length against the simple approach.
- 4 Implement normalised keys for a multi-column sort and measure the speedup against comparator-based sorting.
- 5 Implement block nested loop, index nested loop, sort-merge, and hash joins. Measure I/O for each on the same inputs.
- 6 Find the crossover: at what outer-relation size does hash join overtake index nested loop on your data?
- 7 Construct a non-equi join and confirm that only nested loop applies.
- 8 **Demonstrate the spill cliff:** run a hash join with a work memory setting just large enough, then reduce it by 10% and measure the time difference.
- 9 Construct a skewed join key where one value dominates, and show that recursive partitioning does not help.
- 10 Take a slow query in PostgreSQL, read EXPLAIN ANALYZE, identify the estimation error, and fix it by improving statistics rather than by hinting.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Describe the two phases of external merge sort and say why all I/O is sequential.
- 2 Write the pass-count formula and explain why more memory helps so much.
- 3 Name five refinements real sorts use.

- 4 Give the three nested loop variants with their costs, and say when index nested loop wins.
- 5 When is sort-merge join free, and what else does it provide?
- 6 Explain Grace hash join and why partitioning makes phase 2 one pass.
- 7 Why can hash join not handle inequality predicates?
- 8 Why is spilling a cliff rather than a slope, and what causes it?
- 9 What does skew do to a hash join, and why does repartitioning not help?
- 10 What is the first thing to look at in EXPLAIN ANALYZE, and what should you fix?